PHÂN TÍCH THIẾT KẾ THUẬT TOÁN

Chương 4

CHIẾN LƯỢC CHIA ĐỂ TRỊ

(Divide and Conquer)

GV: Bùi Văn Thượng
Email: thuong.bui@ut.edu.vn

NỘI DUNG CHÍNH

Cơ sở của chiến lược chia để trị

Sắp xếp trộn (Merge sort)

Sắp xếp nhanh (Quick sort)

Tìm kiếm nhị phân

Cơ sở của chiến lược chia để trị

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Chiến lược Chia để trị (Divide and Conquer)

Đây là "vũ khí" hạng nặng trong khoa học máy tính, là cơ sở để tạo ra những thuật toán có độ phức tạp tối ưu như $O(n \log n)$ hay $O(\log n)$.

Đúng như tên gọi của nó, chiến lược này dựa trên một tư duy quản trị kinh điển: Một vấn đề lớn nếu quá khó để giải quyết ngay lập tức, hãy chia nhỏ nó ra cho đến khi những phần nhỏ đó trở nên tầm thường.

Thay vì xử lý một mảng 1 triệu phần tử, ta xử lý 2 mảng nửa triệu phần tử, rồi tiếp tục chia cho đến khi chỉ còn 1 phần tử (lúc này bài toán đã có lời giải hiển nhiên).

Cơ sở của chiến lược chia để trị

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Quy trình 3 bước cốt lõi

- Bước 1: Chia (Divide)

  Chia bài toán ban đầu (P) thành các bài toán con (P_1, P_2, ..., P_k) có quy mô nhỏ hơn. Thông thường, các bài toán con này có cùng dạng với bài toán gốc.

- Bước 2: Trị (Conquer)

  Giải quyết các bài toán con bằng cách gọi đệ quy. Trường hợp cơ sở (Base Case): Nếu bài toán con đủ nhỏ, giải nó một cách trực tiếp.

- Bước 3: Kết hợp (Combine)

  Tổng hợp lời giải của các bài toán con để thu được lời giải cho bài toán ban đầu. Đây thường là bước quan trọng nhất và quyết định hiệu năng của thuật toán (ví dụ: bước Merge trong Merge Sort).

Cơ sở của chiến lược chia để trị

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Mô hình toán học (Hàm truy hồi)

Để phân tích thời gian chạy của chiến lược này, các nhà khoa học máy tính sử dụng phương trình truy hồi có dạng tổng quát:

$$T(n) = aT(n/b) + f(n)$$

Trong đó:

- n: Kích thước bài toán gốc.
- a: Số lượng bài toán con được tạo ra mỗi bước.
- n/b: Kích thước của mỗi bài toán con.
- f(n): Chi phí để Chia bài toán và Kết hợp kết quả.

Cơ sở của chiến lược chia để trị

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Các ứng dụng kinh điển

Chiến lược Chia để trị sinh ra những giải thuật làm thay đổi hiệu năng của phần mềm:

- Tìm kiếm: Binary Search (Chia đôi vùng tìm kiếm).
- Sắp xếp: Merge Sort, Quick Sort.
- Toán học: Nhân ma trận Strassen, Nhân số nguyên lớn Karatsuba, Biến đổi Fourier nhanh (FFT).
- Hình học tính toán: Tìm cặp điểm gần nhất.

Cơ sở của chiến lược chia để trị

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ưu điểm:

- Hiệu năng vượt trội: Chuyển đổi từ $O(n^2)$ xuống $O(n \log n)$ giúp xử lý dữ liệu lớn trong thực tế.
- Tận dụng đa nhân (Parallelism): Các bài toán con độc lập với nhau nên có thể chạy song song trên nhiều nhân CPU/GPU cùng lúc.
- Cấu trúc rõ ràng: Code thường ngắn gọn và mang tính học thuật cao.

Nhược điểm:

- Tốn bộ nhớ Stack: Do sử dụng đệ quy sâu.
- Chi phí kết hợp: Nếu bước "Combine" quá phức tạp, nó có thể triệt tiêu lợi ích của việc chia nhỏ.

Sắp xếp trộn (Merge Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Nguyên lý hoạt động của Merge Sort

Merge Sort áp dụng triệt để mô hình 3 bước của Chia để trị:

1. Chia (Divide): Chia mảng hiện tại thành hai mảng con bằng cách tìm điểm chính giữa. Quá trình này lặp lại cho đến khi mỗi mảng con chỉ còn đúng 1 phần tử (mảng 1 phần tử mặc nhiên đã được sắp xếp).
2. Trị (Conquer): Đệ quy tiếp tục chia để sắp xếp các mảng con.
3. Kết hợp (Combine - Trộn): Đây là bước quan trọng nhất. Trộn hai mảng con đã sắp xếp để tạo thành một mảng mới lớn hơn cũng đã được sắp xếp.

Sắp xếp trộn (Merge Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Giải thuật Trộn (The Merge Procedure)

Giả sử ta có hai mảng con L và R:

- Ta dùng hai "con trỏ" bắt đầu từ đầu mỗi mảng.
- So sánh từng cặp phần tử tại con trỏ. Phần tử nào nhỏ hơn sẽ được đưa vào mảng kết quả và con trỏ tương ứng sẽ nhích lên một vị trí.
- Lặp lại cho đến khi một trong hai mảng hết phần tử, sau đó đưa toàn bộ phần còn lại của mảng kia vào kết quả.

Sắp xếp trộn (Merge Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

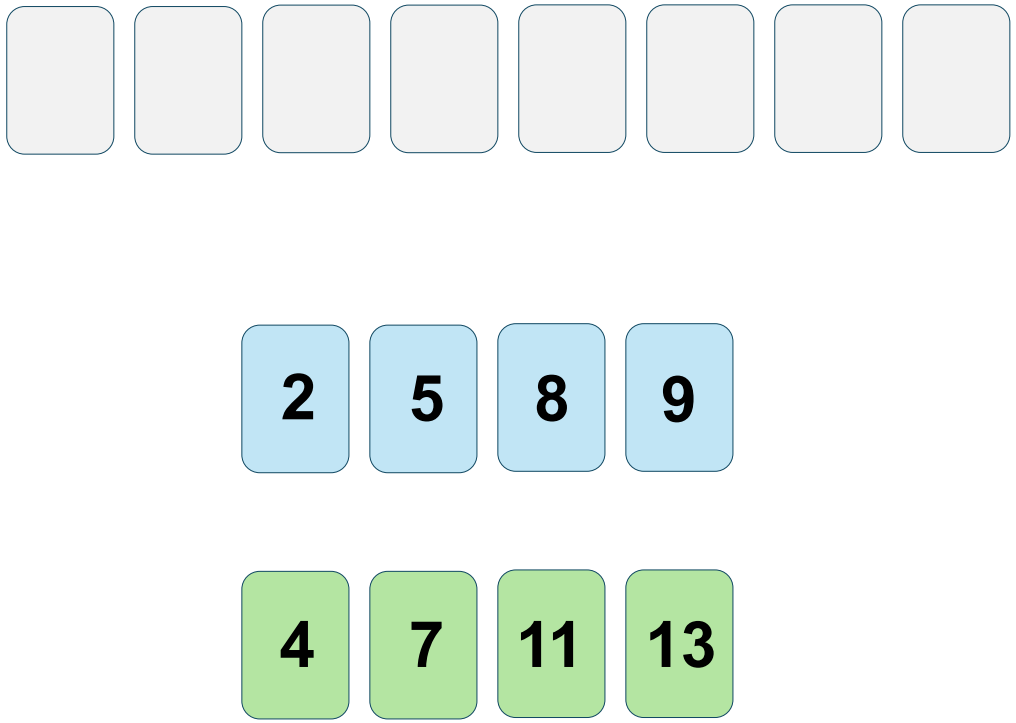
Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



Sắp xếp trộn (Merge Sort)

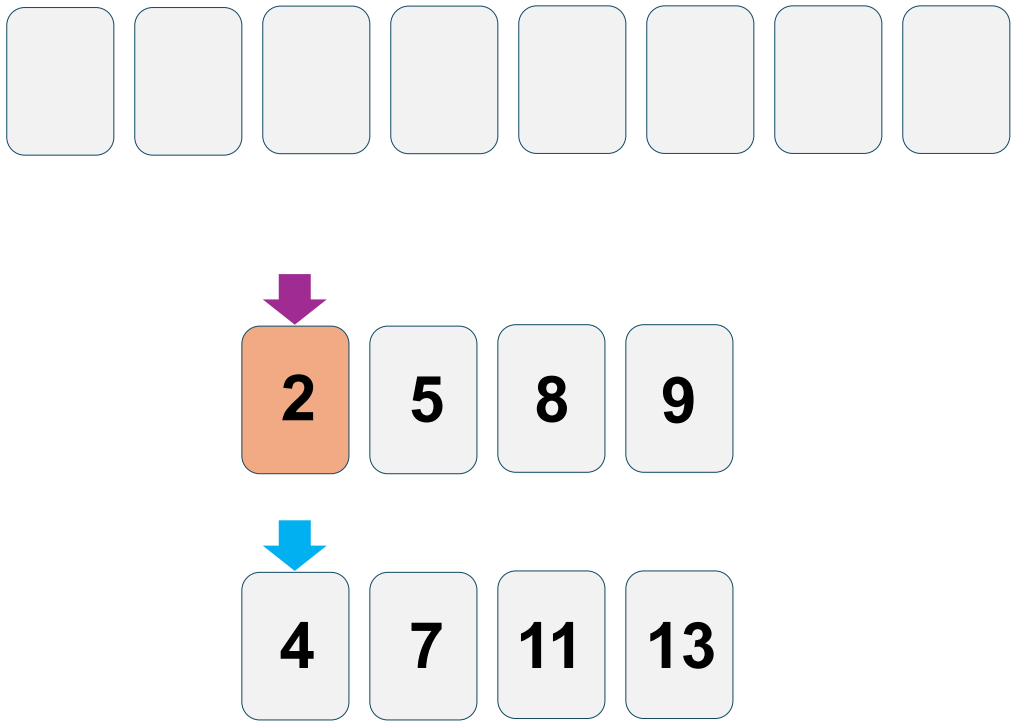
UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



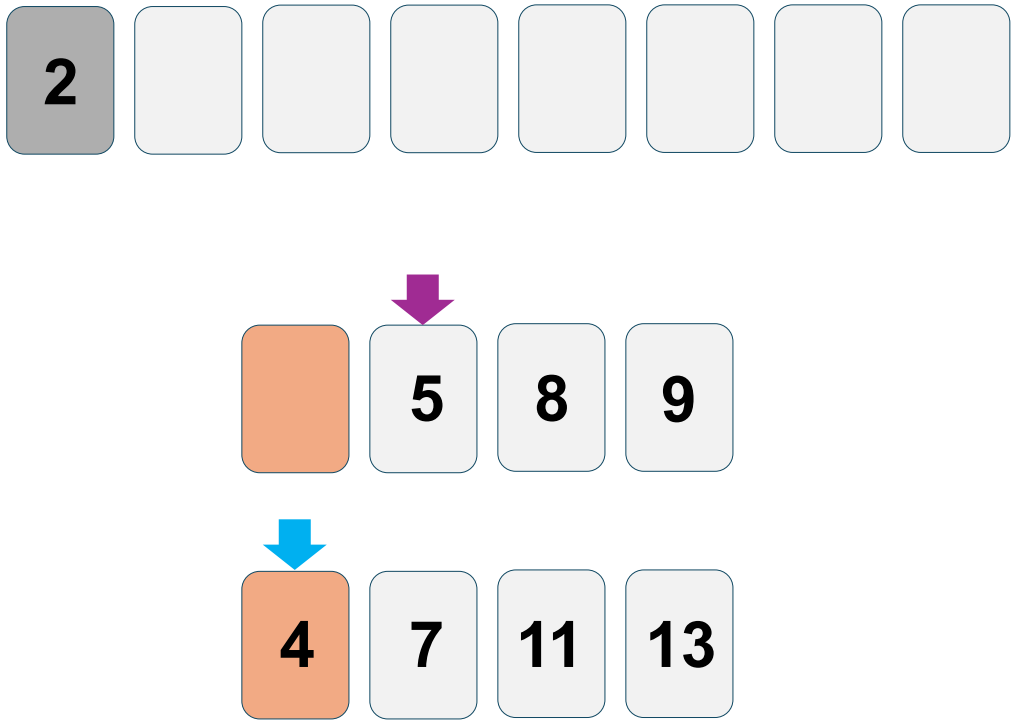
Sắp xếp trộn (Merge Sort)

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



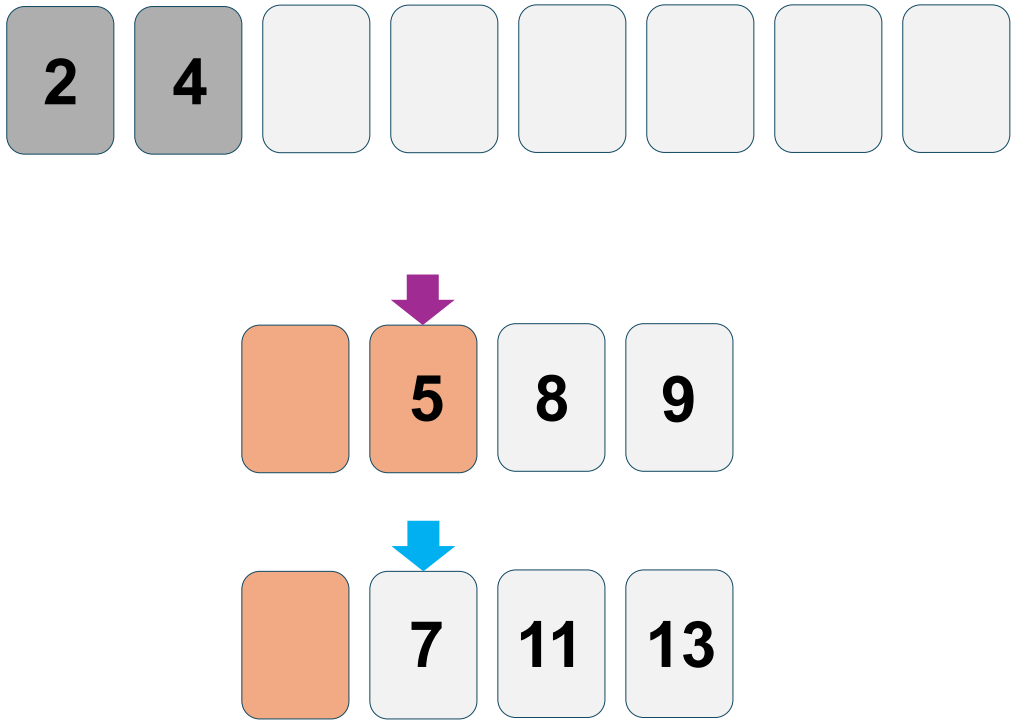
Sắp xếp trộn (Merge Sort)

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



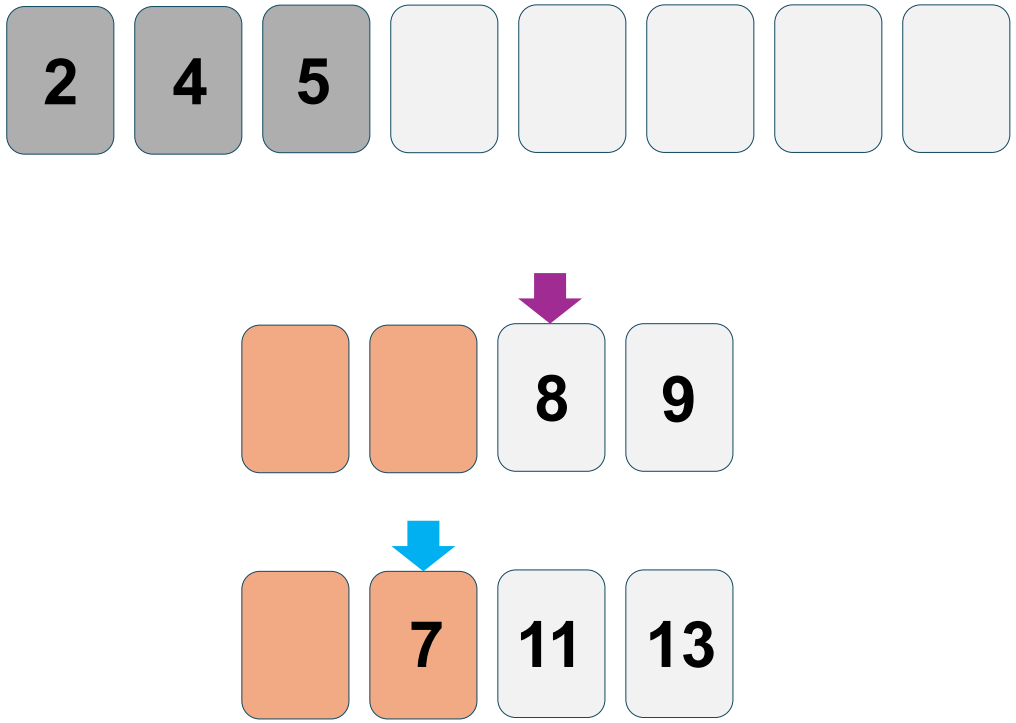
Sắp xếp trộn (Merge Sort)

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



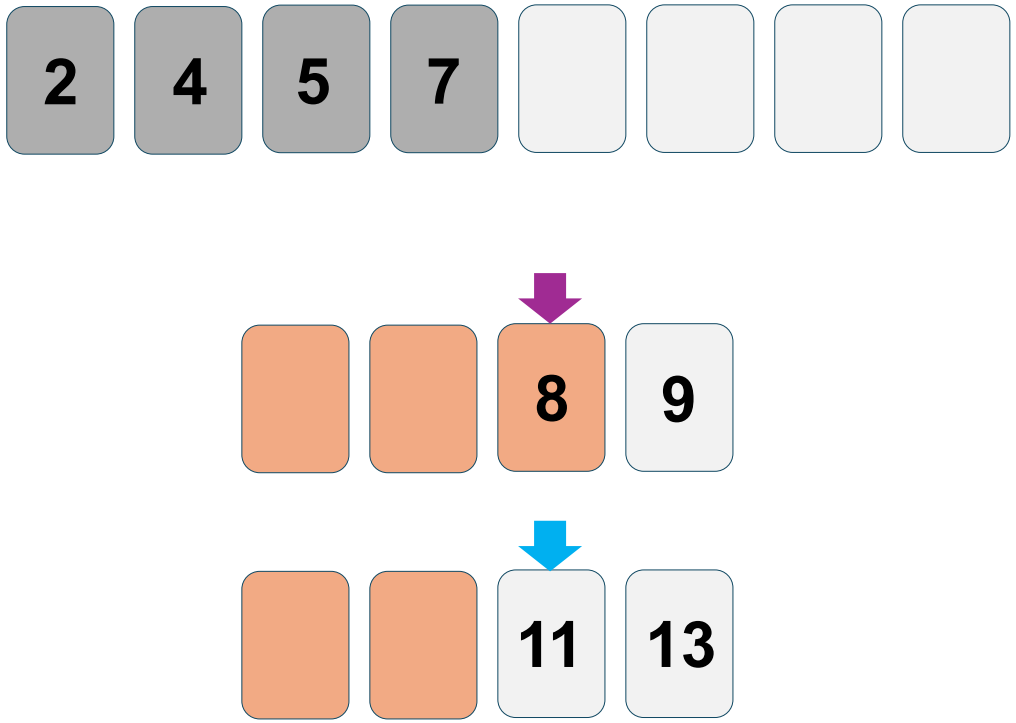
Sắp xếp trộn (Merge Sort)

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



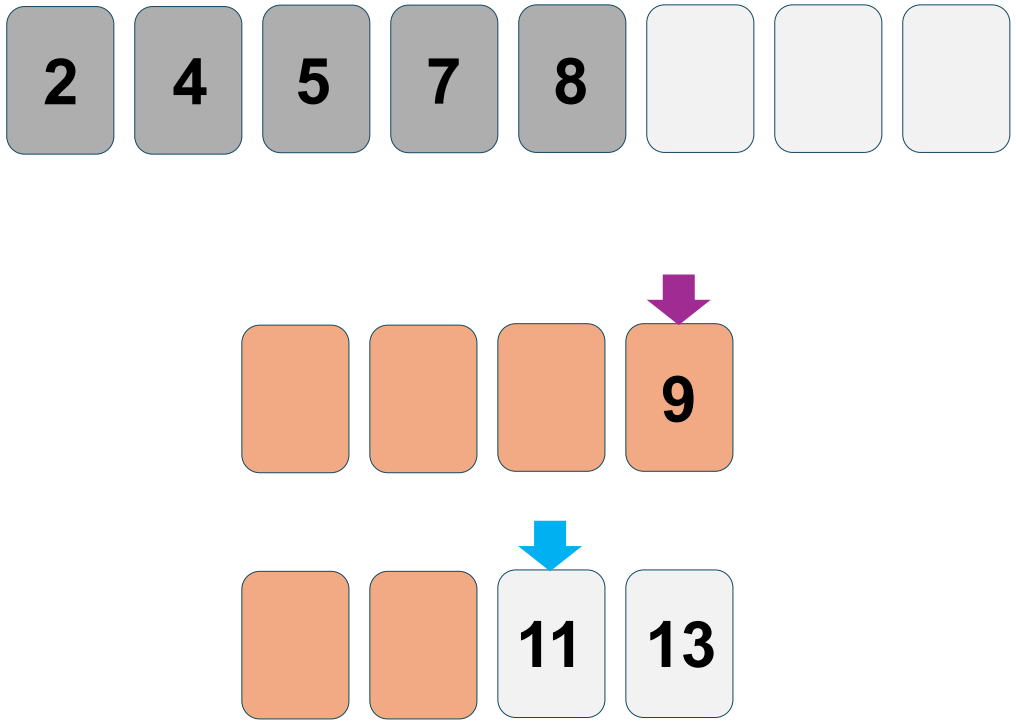
Sắp xếp trộn (Merge Sort)

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



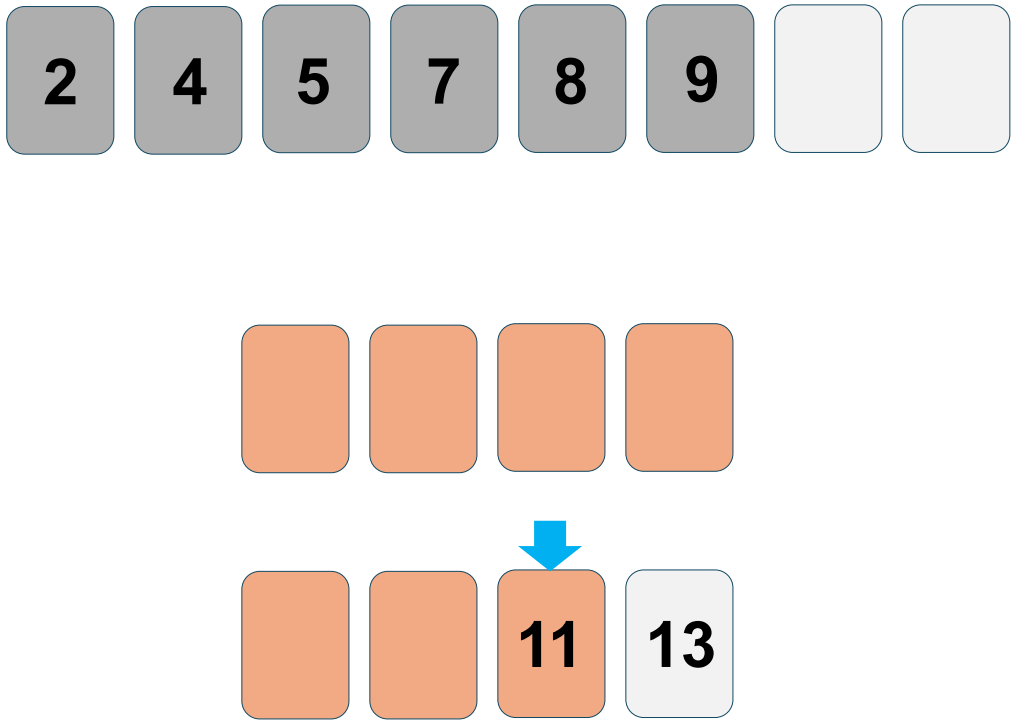
Sắp xếp trộn (Merge Sort)

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



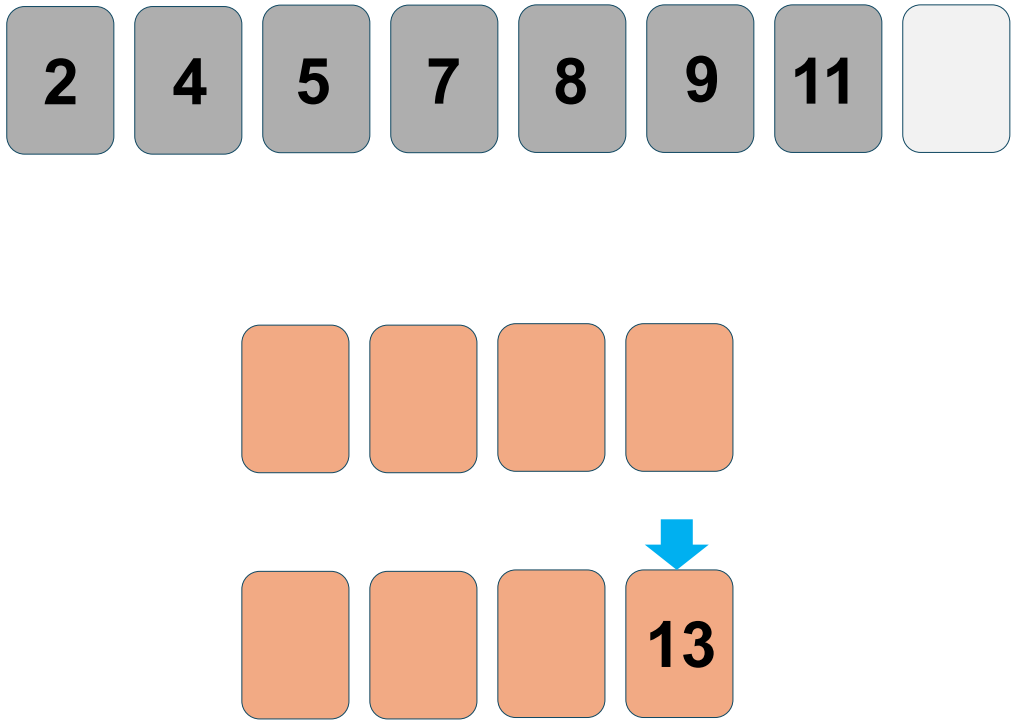
Sắp xếp trộn (Merge Sort)

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



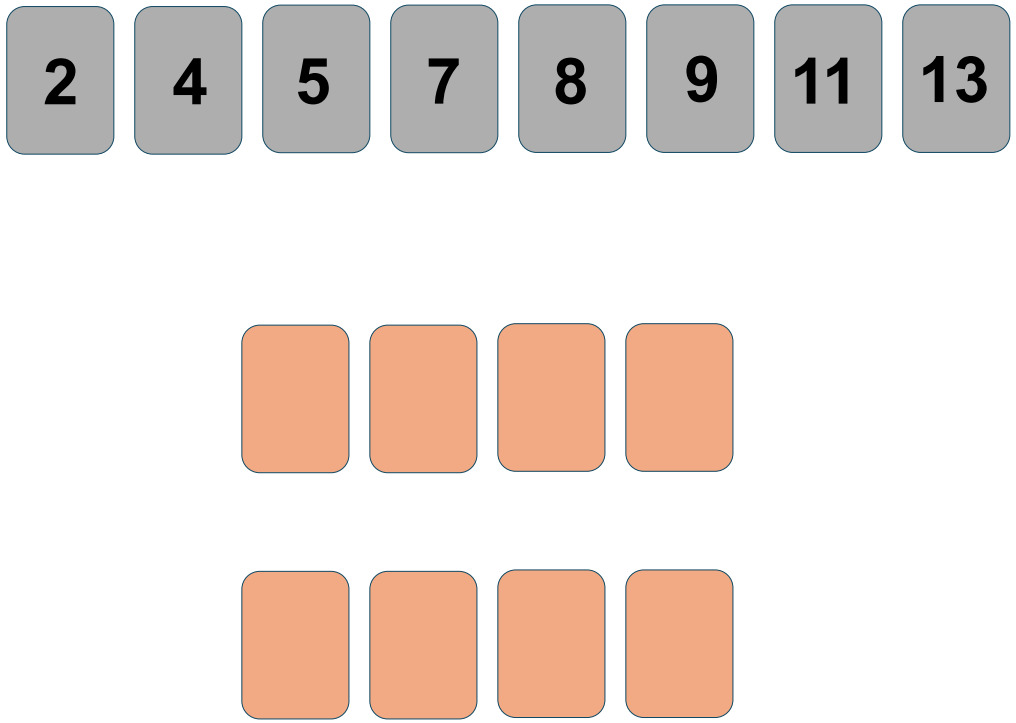
Sắp xếp trộn (Merge Sort)

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



Sắp xếp trộn (Merge Sort)

Ví dụ: Trộn 2 mảng con liên tục đã sắp xếp thành mảng đã sắp xếp.



Sắp xếp trộn (Merge Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Cài đặt giải thuật trộn 2 mảng con liên tục của mảng a bằng C++

```
void merge(int a[], int left, int mid, int right){
    int n1 = mid - left + 1, n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = a[left + i];
    for (int j = 0; j < n2; j++) R[j] = a[mid + 1 + j];
    // Trộn hai mảng tạm vào mảng chính
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            a[k++] = L[i++];
        } else {
            a[k++] = R[j++];
        }
    }
    // Sao chép các phần tử còn lại (nếu có)
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
}
```

Sắp xếp trộn (Merge Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Cài đặt giải thuật Merge Sort bằng C++

```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);      // Chia để trị nửa trái
        mergeSort(arr, mid + 1, right); // Chia để trị nửa phải

        merge(arr, left, mid, right);   // Kết hợp (Trộn)
    }
}
```

Sắp xếp trộn (Merge Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Phân tích chi tiết (Analysis)

Độ phức tạp thời gian

$O(n \log n)$ cho cả trường hợp tốt nhất, trung bình và xấu nhất.

Độ phức tạp không gian O(n).

Merge Sort không phải là thuật toán sắp xếp tại chỗ (In-place). Nó cần thêm bộ nhớ tạm để lưu các mảng con trong quá trình trộn.

Sắp xếp trộn (Merge Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Các đặc tính quan trọng trong thực tế

- Tính ổn định (Stability): Merge Sort là thuật toán sắp xếp ổn định. Nếu hai phần tử có giá trị bằng nhau, thứ tự tương đối của chúng sẽ được giữ nguyên. Điều này cực kỳ quan trọng khi sắp xếp các đối tượng phức tạp (như danh sách sinh viên theo tên, sau đó theo lớp).
- Sắp xếp ngoài (External Sorting): Vì Merge Sort xử lý dữ liệu theo từng khối nhỏ rồi trộn lại, nó là "ông vua" trong việc sắp xếp các file dữ liệu khổng lồ không thể nạp hết vào RAM.
- Song song hóa: Bước chia bài toán của Merge Sort rất dễ để triển khai chạy song song trên nhiều luồng (Multi-threading).

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Nguyên lý hoạt động: Chiến lược "Phân hoạch“

Quick Sort xoay quanh một khái niệm then chốt gọi là Phần tử chốt (Pivot).

- Chia (Divide): Chọn một phần tử bất kỳ trong mảng làm Pivot. Sau đó, thực hiện Phân hoạch (Partitioning): đưa tất cả các phần tử nhỏ hơn Pivot sang bên trái và tất cả các phần tử lớn hơn Pivot sang bên phải. Lúc này, Pivot nằm đúng vị trí cuối cùng của nó trong mảng đã sắp xếp.
- Trị (Conquer): Đệ quy áp dụng quy trình trên cho hai mảng con bên trái và bên phải của Pivot.
- Kết hợp (Combine): Khác với Merge Sort, Quick Sort không cần bước kết hợp. Khi các mảng con đã được sắp xếp tại chỗ, toàn bộ mảng ban đầu đã hoàn thành.

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Kỹ thuật Phân hoạch (Partitioning)

Đây là "linh hồn" của thuật toán. Có hai sơ đồ phân hoạch phổ biến:

- Lomuto Partition: Dễ hiểu, thường dùng trong giảng dạy.
- Hoare Partition: Hiệu quả hơn, thực hiện ít phép hoán đổi hơn.

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Mã giả sơ đồ Lomuto:

- Chọn phần tử cuối cùng làm Pivot.
- Dùng một biến i để đánh dấu biên giới của nhóm nhỏ hơn Pivot.
- Duyệt mảng, nếu thấy phần tử nào nhỏ hơn Pivot, hoán đổi nó vào vị trí i và tăng i.
- Cuối cùng, đưa Pivot vào vị trí i.

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Cài đặt Lomuto bằng C++:

```
// Hàm phân hoạch theo sơ đồ Lomuto
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Chọn phần tử cuối làm chốt
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return (i + 1);
}
```

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Cài đặt Lomuto bằng C++:

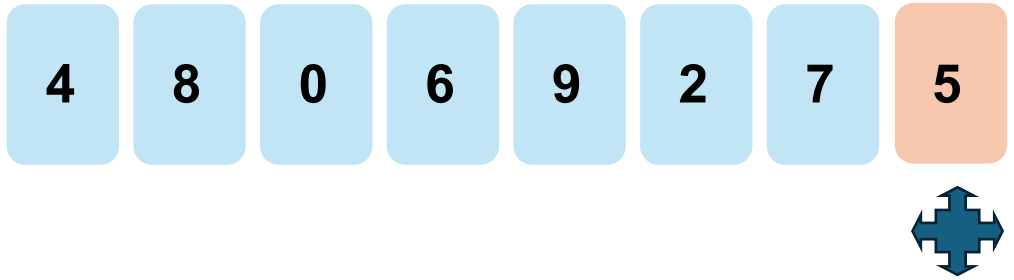
```
// Hàm đệ quy Quick Sort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // pi là chỉ số phân hoạch, arr[pi] đã đúng vị trí
        int pi = partition(arr, low, high);

        // Sắp xếp độc lập hai nửa
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Sắp xếp nhanh (Quick Sort)

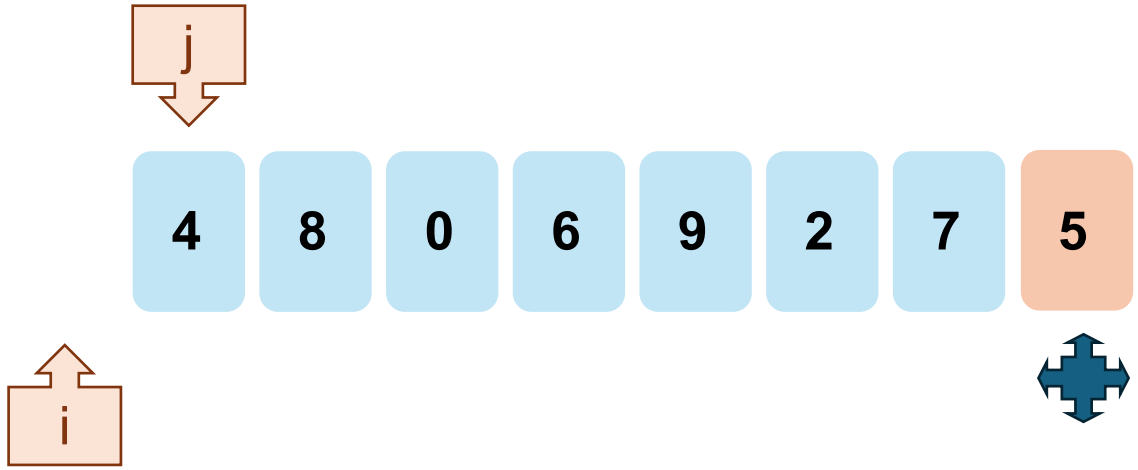
UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



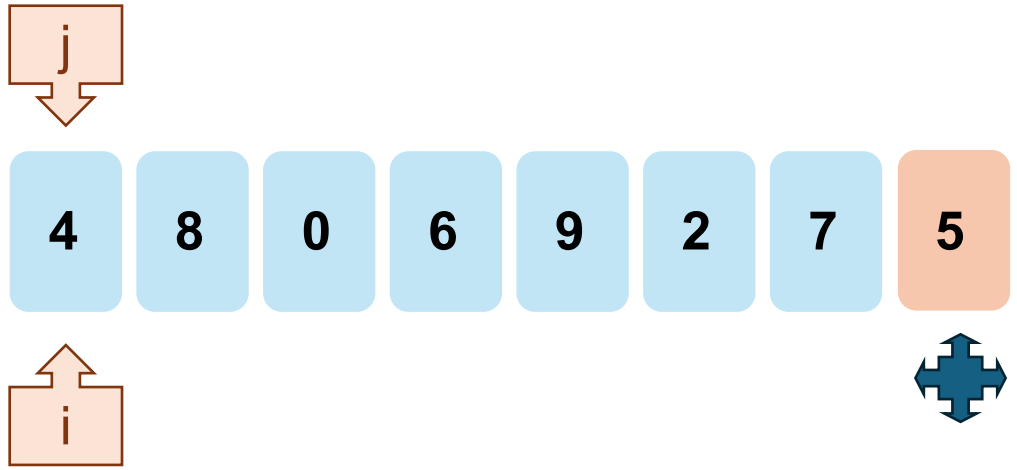
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



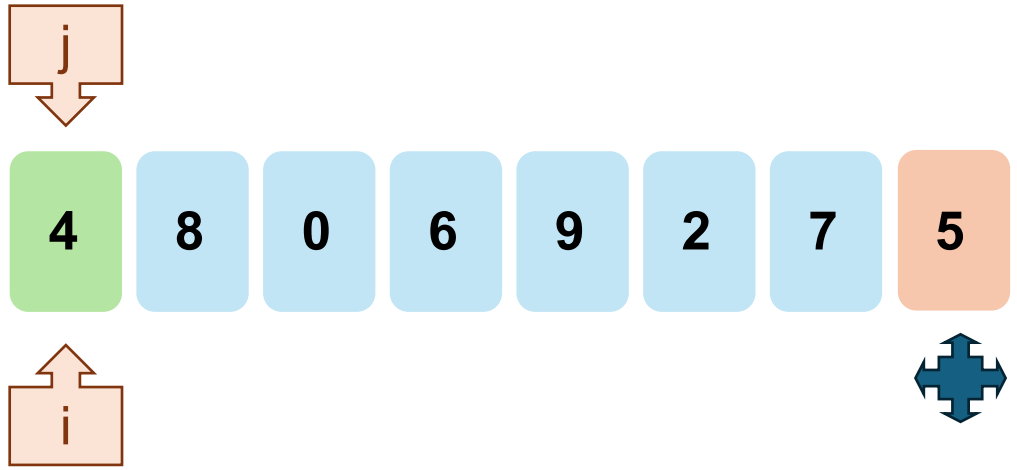
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



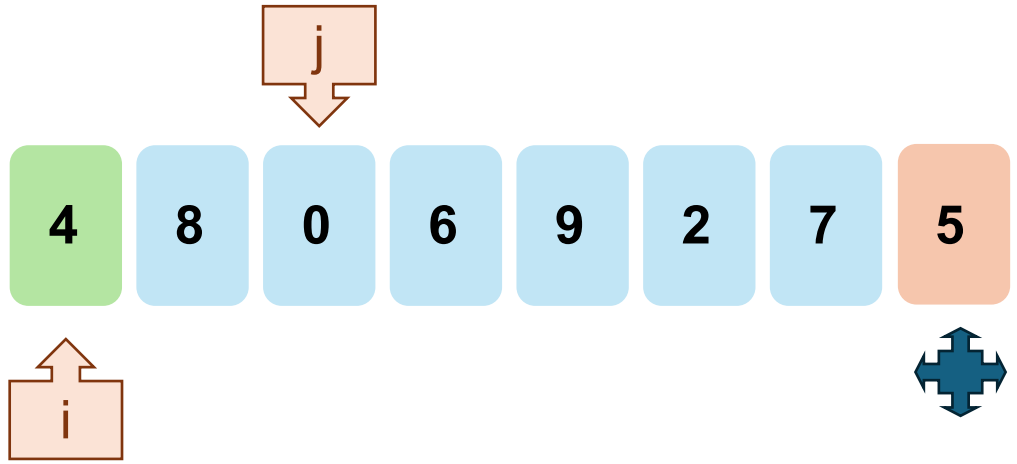
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



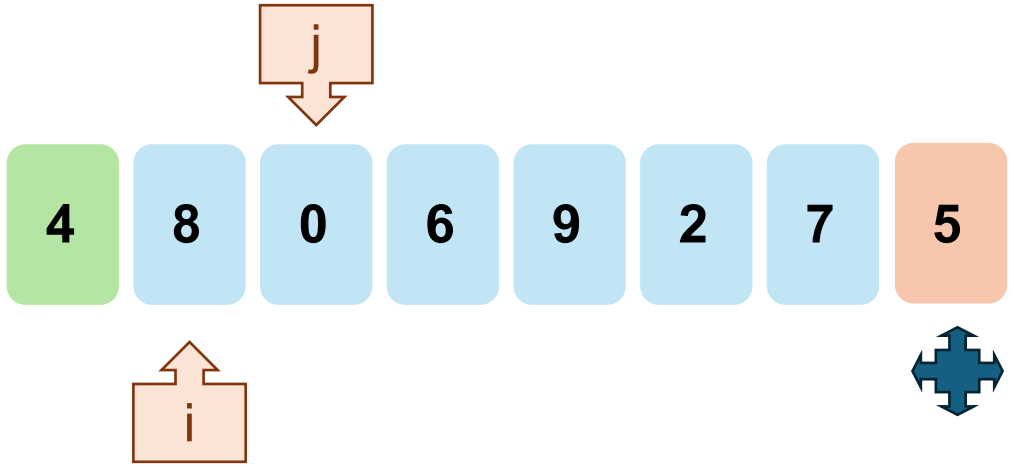
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



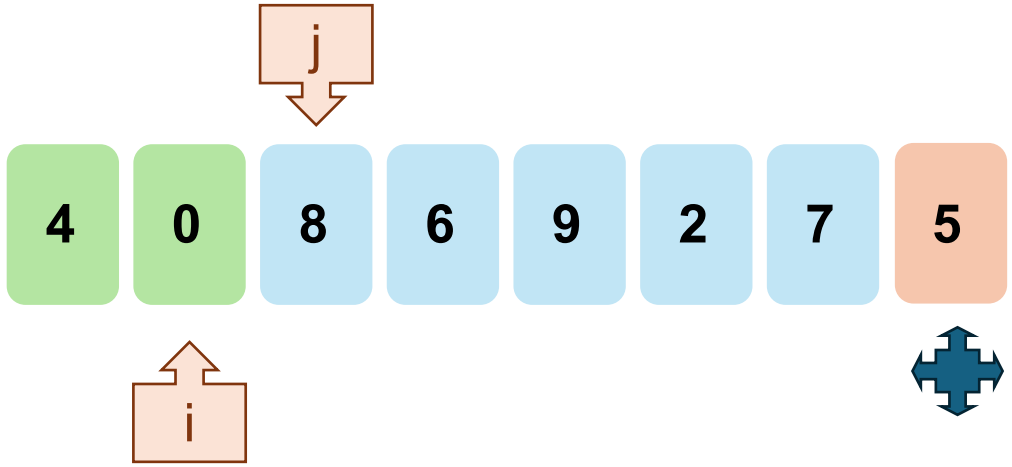
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



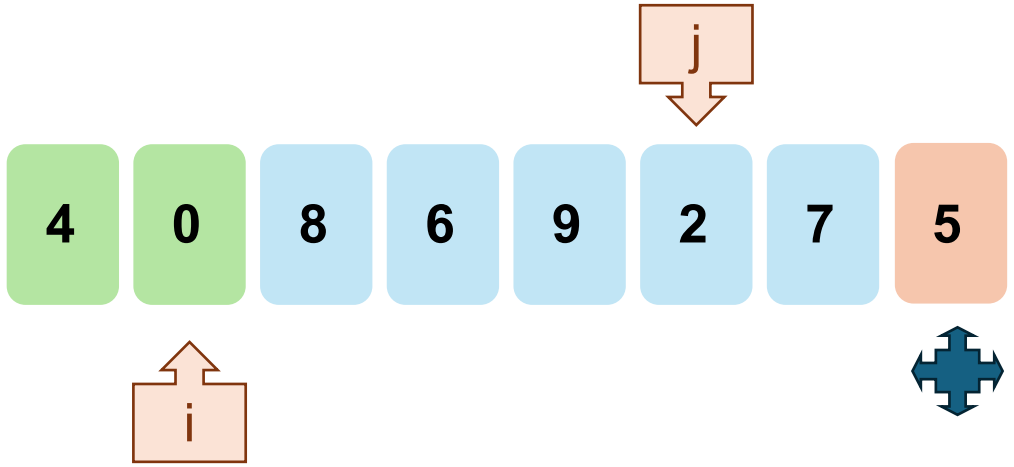
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



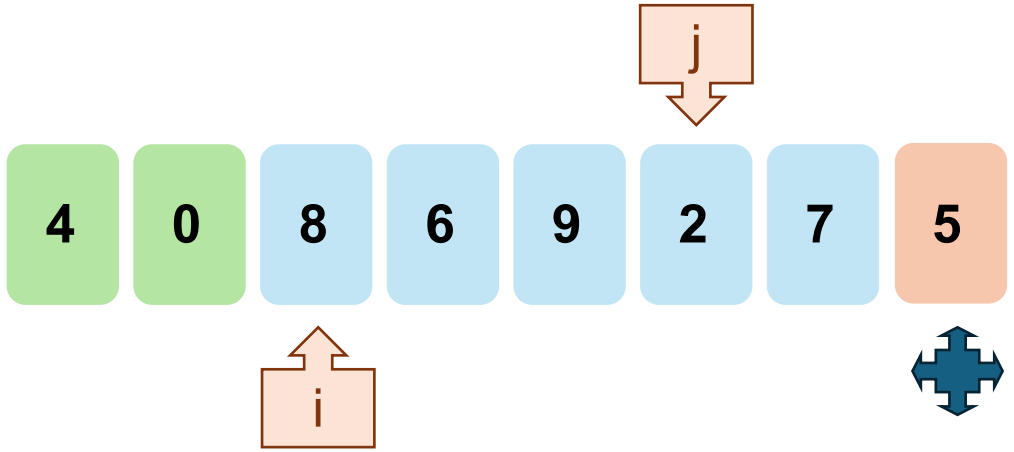
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



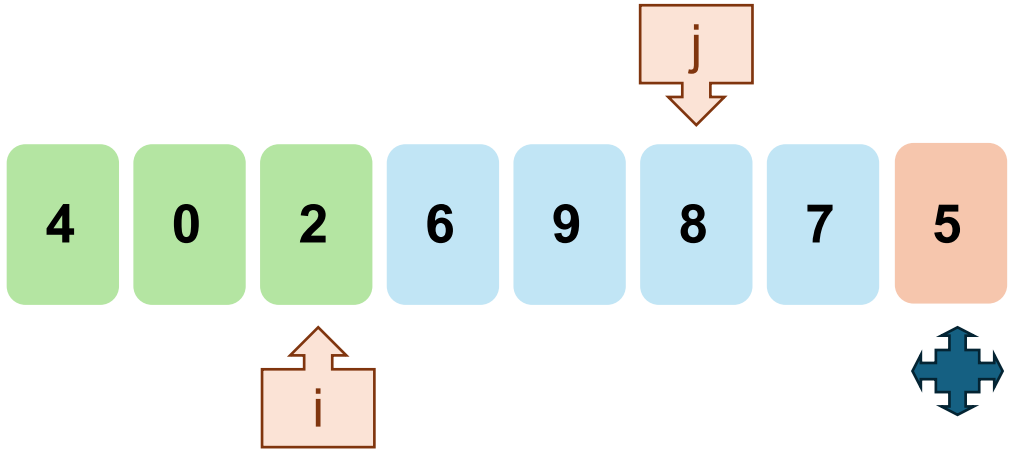
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



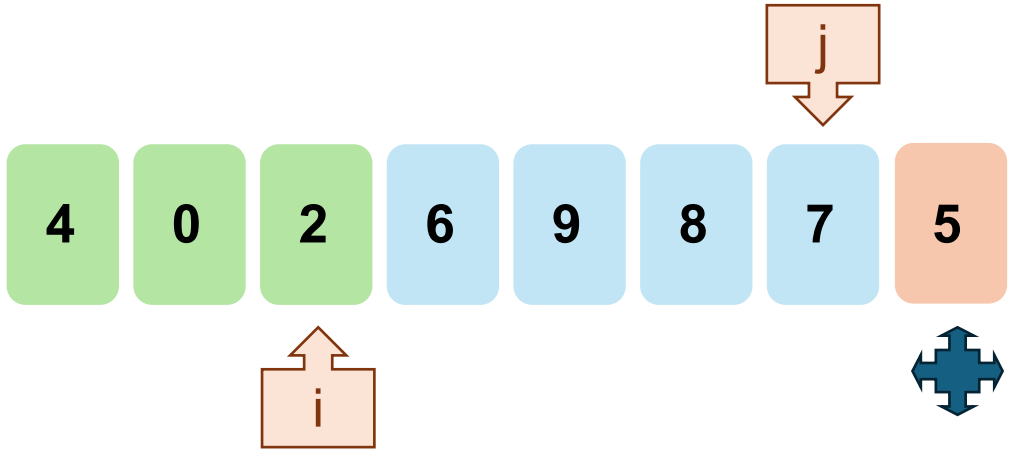
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



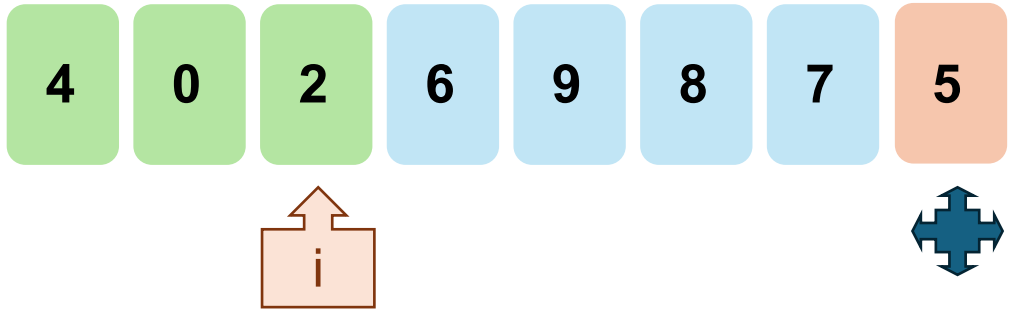
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



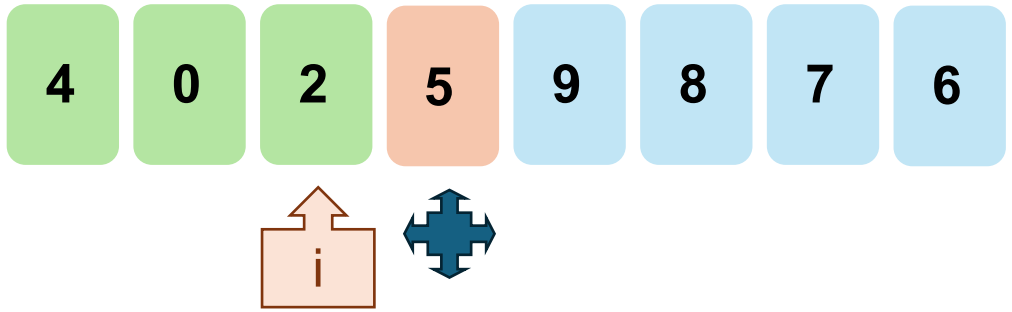
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



Sắp xếp nhanh (Quick Sort)

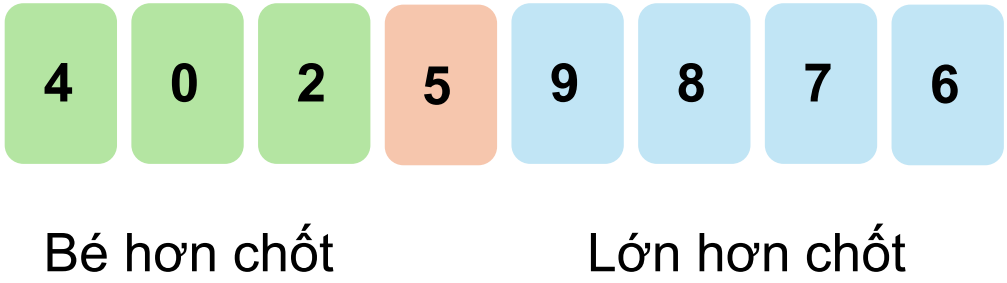
Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



Đổi giá trị vị trí i+1 với vị trí chốt

Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân hoạch mảng sau với phần tử chốt nằm cuối mảng.



Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Nguyên lý hoạt động của Hoare Partition

Khác với Lomuto sử dụng một con trỏ chạy từ trái sang phải, Hoare sử dụng hai con trỏ bắt đầu từ hai đầu của mảng và di chuyển hướng vào nhau.

- Chọn Pivot: Thường chọn phần tử đầu tiên (arr[low]).
- Con trỏ trái (i): Bắt đầu từ đầu mảng, tăng dần cho đến khi gặp phần tử ≥ Pivot.
- Con trỏ phải (j): Bắt đầu từ cuối mảng, giảm dần cho đến khi gặp phần tử ≤ Pivot.
- Hoán đổi: Nếu hai con trỏ chưa chạm nhau (i < j), ta hoán đổi arr[i] và arr[j], sau đó tiếp tục bước 2 và 3. Nếu hai con trỏ đã gặp nhau hoặc vượt qua nhau, ta trả về j làm điểm phân hoạch.

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Cài đặt mã nguồn C++ của Hoare Partition

```
int hoarePartition(int arr[], int low, int high) {
    int pivot = arr[low]; // Chọn phần tử đầu làm Pivot
    int i = low - 1;
    int j = high + 1;
    while (true) {
        // Tìm phần tử bên trái lớn hơn hoặc bằng Pivot
        do {
            i++;
        } while (arr[i] < pivot);
        // Tìm phần tử bên phải nhỏ hơn hoặc bằng Pivot
        do {
            j--;
        } while (arr[j] > pivot);
        // Nếu hai con trỏ gặp nhau
        if (i >= j) return j;
        // Hoán đổi hai phần tử sai vị trí
        swap(arr[i], arr[j]);
    }
}
```

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Cài đặt mã nguồn C++ của Hoare Partition

```
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = hoarePartition(arr, low, high);

        // Lưu ý: Nửa trái là từ low đến p, nửa phải là từ p + 1 đến high
        quickSort(arr, low, p);
        quickSort(arr, p + 1, high);
    }
}
```

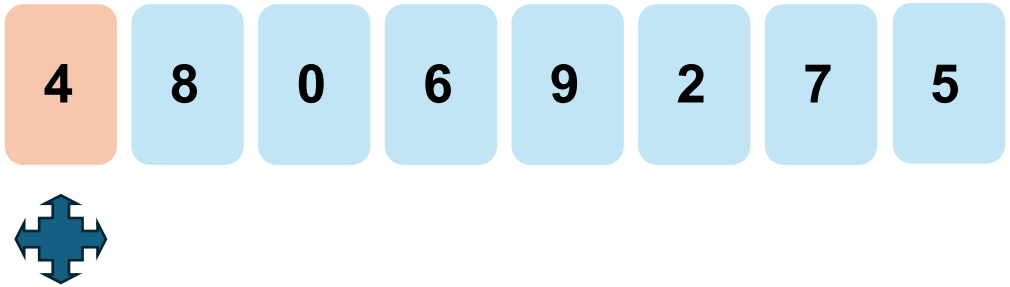
Lưu ý cực kỳ quan trọng khi cài đặt

Khi sử dụng Hoare Partition, cấu trúc của hàm quickSort phải thay đổi một chút so với Lomuto:

Lý do: Khác với Lomuto (phần tử Pivot được đưa về đúng vị trí và nằm ngoài các lần gọi đệ quy tiếp theo), trong Hoare, điểm trả về j chỉ đảm bảo rằng mọi thứ bên trái ≤ mọi thứ bên phải. Bản thân Pivot có thể không nằm chính xác tại vị trí j.

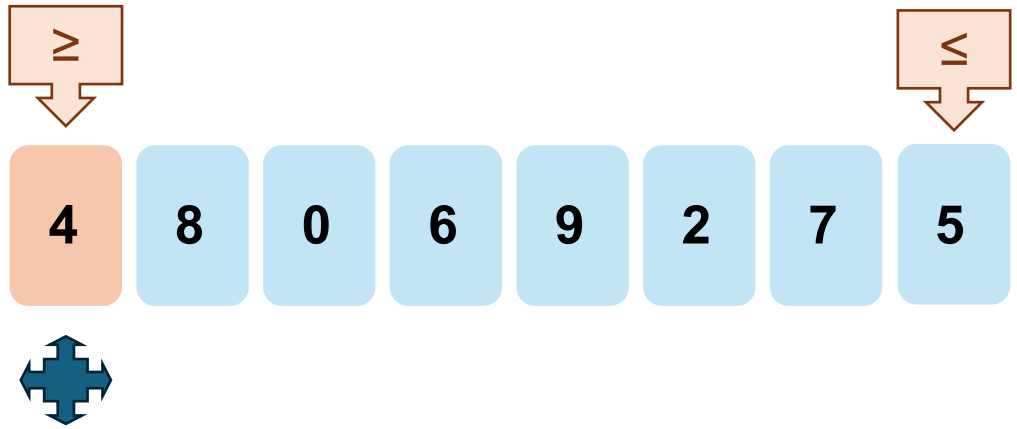
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân đoạn mảng sau với chốt là phần tử đầu mảng.



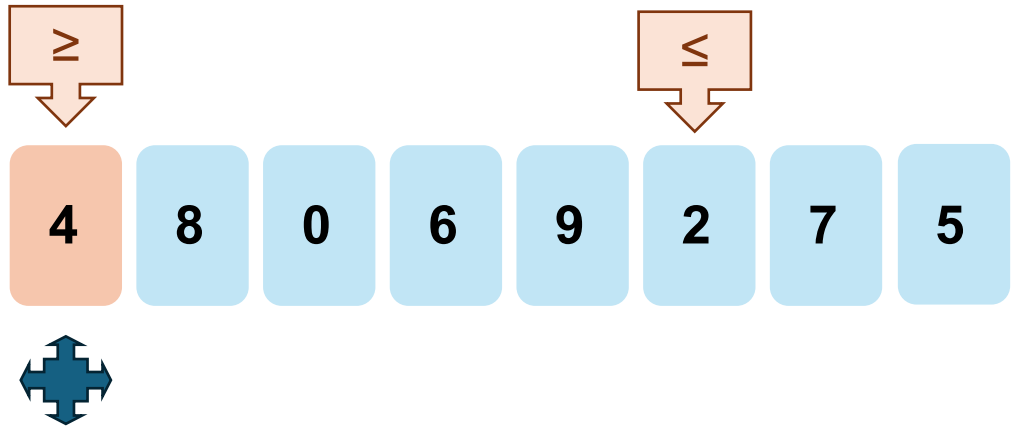
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân đoạn mảng sau với chốt là phần tử đầu mảng.



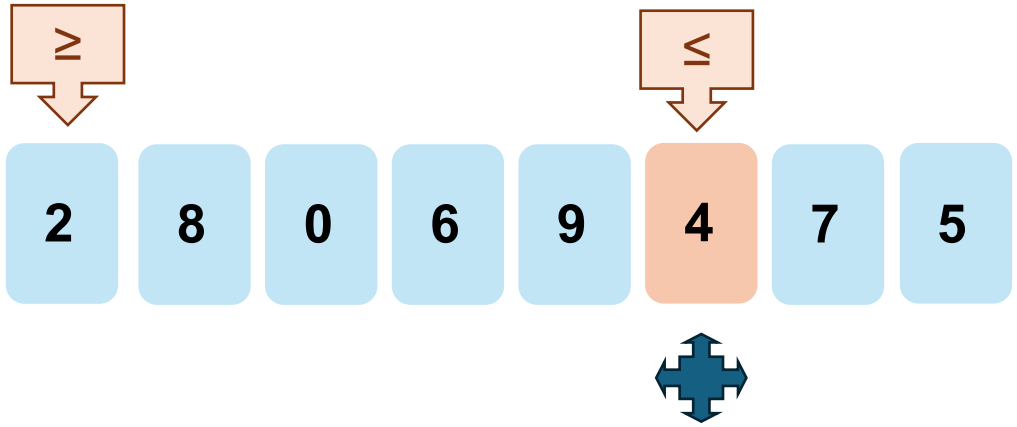
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân đoạn mảng sau với chốt là phần tử đầu mảng.



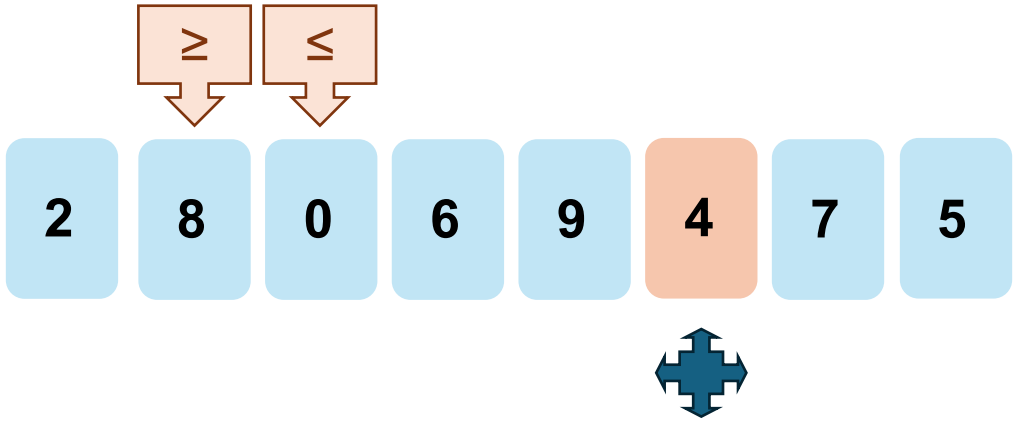
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân đoạn mảng sau với chốt là phần tử đầu mảng.



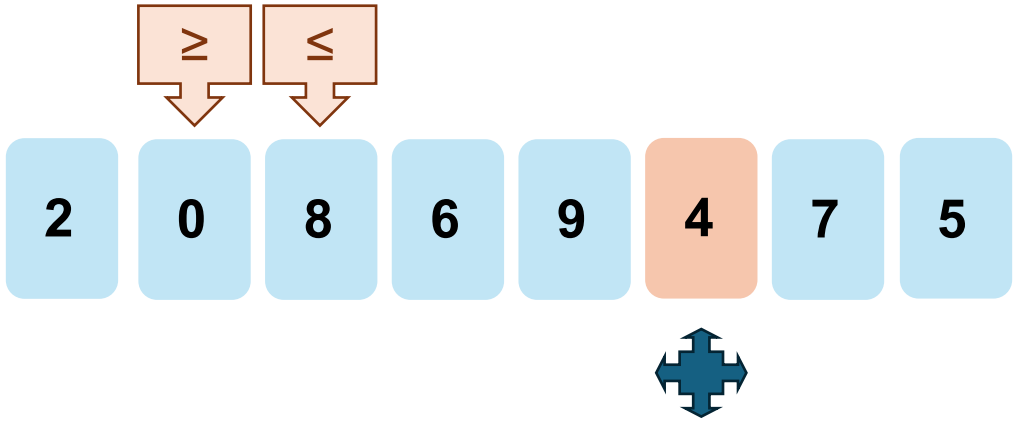
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân đoạn mảng sau với chốt là phần tử đầu mảng.



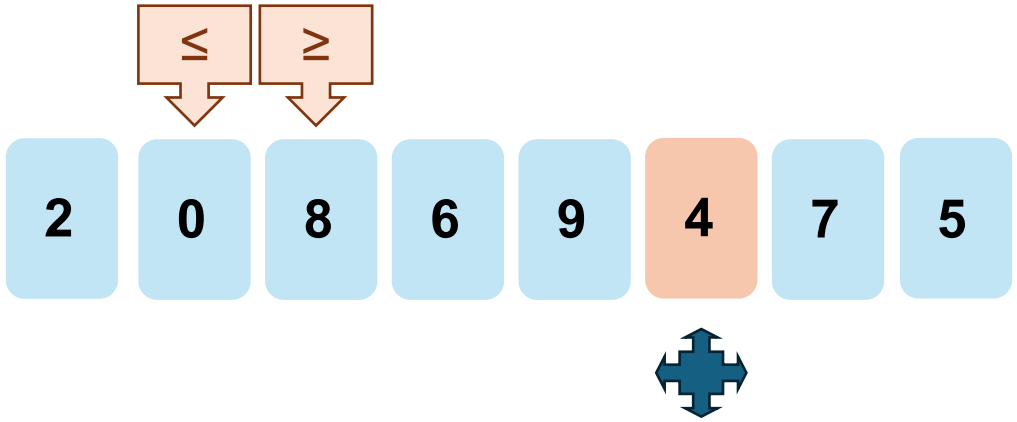
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân đoạn mảng sau với chốt là phần tử đầu mảng.



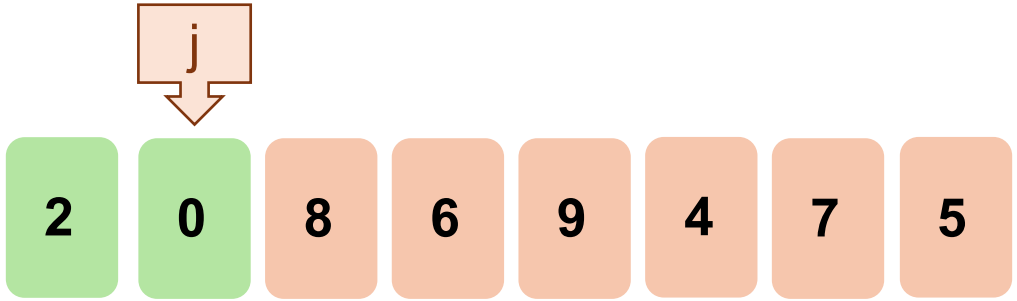
Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân đoạn mảng sau với chốt là phần tử đầu mảng.



Sắp xếp nhanh (Quick Sort)

Ví dụ: Phân đoạn mảng sau với chốt là phần tử đầu mảng.



Giải thuật trả về vị trí điểm phân hoạch là j

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Phân tích hiệu năng

Độ phức tạp của Quick Sort phụ thuộc rất lớn vào cách chọn Pivot.

- Trường hợp tốt nhất: $O(n \log n)$
- Trường hợp trung bình: $O(n \log n)$
- Trường hợp xấu nhất: $O(n^2)$

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Mẹo tối ưu hóa trong thực tế

Để tránh rơi vào trường hợp xấu nhất $O(n^2)$, các kỹ sư thường:

- Chọn Pivot ngẫu nhiên (Randomized Quick Sort).
- Chọn trung vị của 3 phần tử (Đầu, Giữa, Cuối).
- Hybrid Sort: Khi mảng con đủ nhỏ (khoảng 10 phần tử), chuyển sang dùng Insertion Sort để tối ưu tốc độ.

Sắp xếp nhanh (Quick Sort)

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

So sánh Quick Sort và Merge Sort

Đặc điểm	**Quick Sort**	**Merge Sort**
Độ phức tạp trung bình	O(n log n)	O(n log n)
Độ phức tạp xấu nhất	$O(n^2)$	O(n log n)
Bộ nhớ phụ	O(log n) (tại chỗ)	O(n)
Tính ổn định	Không ổn định	Ổn định
Tốt nhất cho	Mảng (Array)	Danh sách liên kết, Dữ liệu lớn

Tìm kiếm nhị phân

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ý tưởng cốt lõi

Khác với tìm kiếm tuyến tính có thể áp dụng cho mọi loại mảng, tìm kiếm nhị phân chỉ hoạt động trên danh sách đã được sắp xếp (tăng dần hoặc giảm dần).

Thay vì kiểm tra từng phần tử từ đầu đến cuối, chúng ta luôn kiểm tra phần tử nằm ở giữa (mid) của danh sách:

- Nếu A[mid] đúng bằng giá trị cần tìm (x): Thành công.
- Nếu x < A[mid]: Vì mảng đã sắp xếp, x chắc chắn chỉ có thể nằm ở nửa bên trái. Ta loại bỏ toàn bộ nửa bên phải.
- Nếu x > A[mid]: x chắc chắn chỉ có thể nằm ở nửa bên phải. Ta loại bỏ toàn bộ nửa bên trái.

Quá trình này lặp lại cho đến khi tìm thấy x hoặc vùng tìm kiếm không còn phần tử nào.

Tìm kiếm nhị phân

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Các bước thực hiện chi tiết

Giả sử tìm x trong mảng A đã sắp xếp tăng dần, kích thước n:

- Xác định chỉ số giữa mid của đoạn đang xét.
- So sánh A[mid] với giá trị cần tìm key.
  - Nếu A[mid] = key → tìm thấy.
  - Nếu A[mid] > key → tìm trong nửa trái.
  - Nếu A[mid] < key → tìm trong nửa phải.
- Lặp lại quá trình cho đến khi tìm thấy hoặc đoạn tìm kiếm rỗng.

Tìm kiếm nhị phân

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Đánh giá giải thuật

- Độ phức tạp thời gian: tốt nhất O(*1*), xấu nhất O(log *n*).
- Độ phức tạp không gian: O(*1*).
- Đánh giá chung:
  - Tốc độ tìm kiếm nhanh, phù hợp cho tập dữ liệu lớn.
  - Cần dữ liệu có thứ tự → thường kết hợp với các thuật toán sắp xếp.
  - Được sử dụng rộng rãi trong hệ thống tìm kiếm, cơ sở dữ liệu, xử lý tín hiệu.

Tìm kiếm nhị phân

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Cài đặt mã nguồn (C++)

```
int binarySearch(int A[], int n, int x) {
    int left = 0;
    int right = n - 1;

    while (left <= right) {
        // Cách tính mid an toàn để tránh tràn số int
        int mid = left + (right - left) / 2;

        if (A[mid] == x)
            return mid; // Tìm thấy
        if (A[mid] < x)
            left = mid + 1; // Tìm nửa bên phải
        else
            right = mid - 1; // Tìm nửa bên trái
    }

    return -1; // Không tìm thấy
}
```

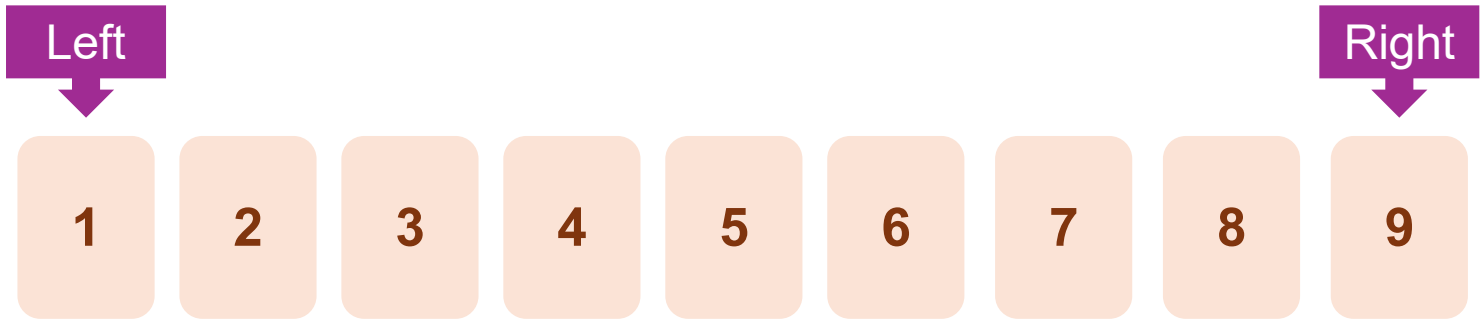
Tìm kiếm nhị phân

Ví dụ: Tìm trong dãy sau phần tử có giá trị là 6



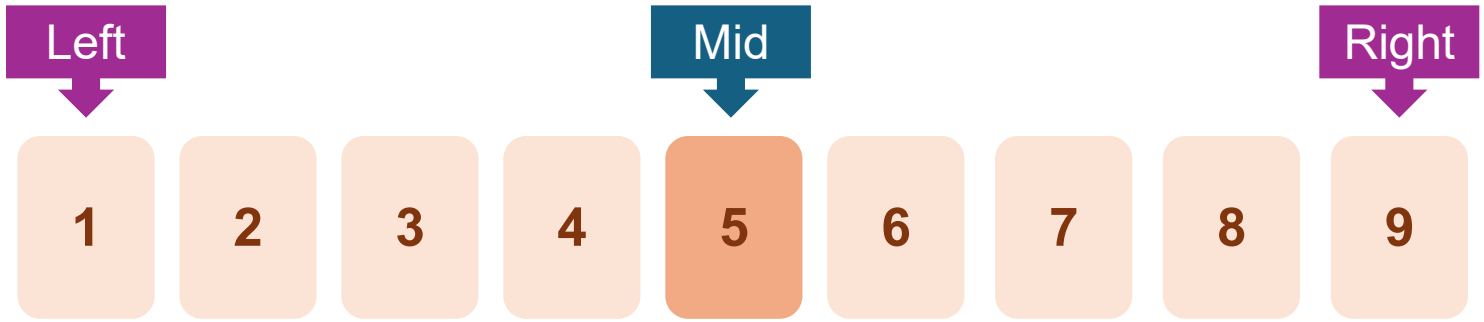
Tìm kiếm nhị phân

Ví dụ: Tìm trong dãy sau phần tử có giá trị là 6



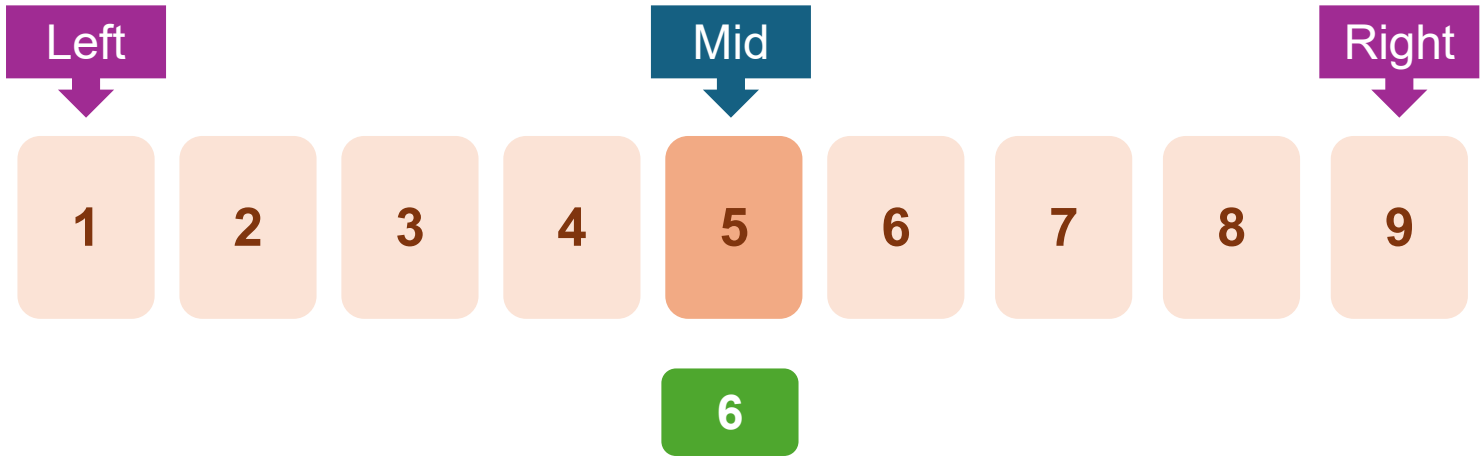
Tìm kiếm nhị phân

Ví dụ: Tìm trong dãy sau phần tử có giá trị là 6



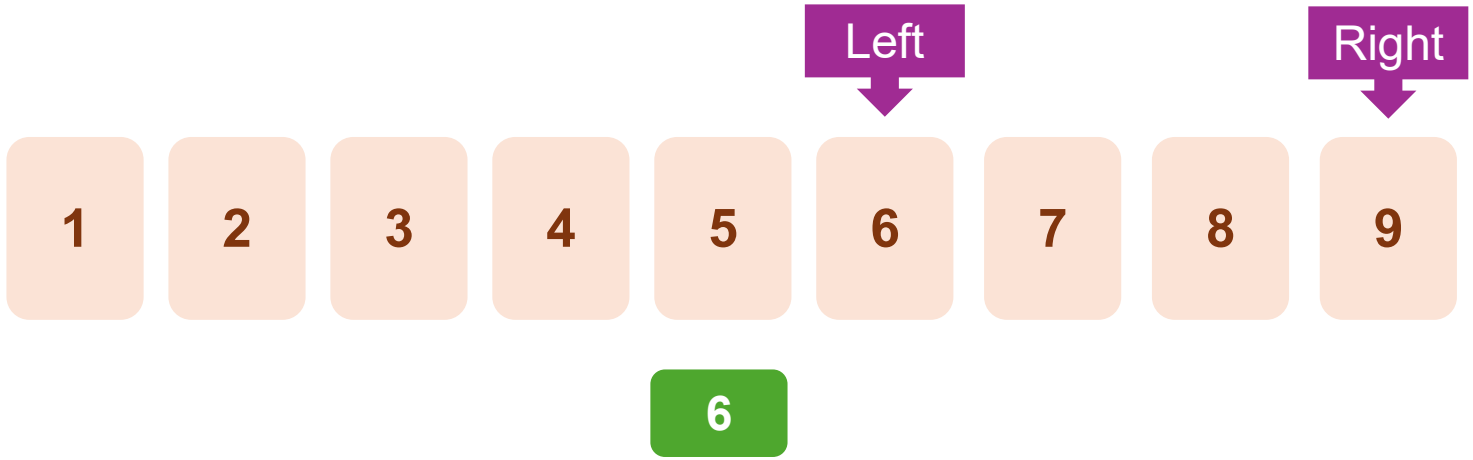
Tìm kiếm nhị phân

Ví dụ: Tìm trong dãy sau phần tử có giá trị là 6



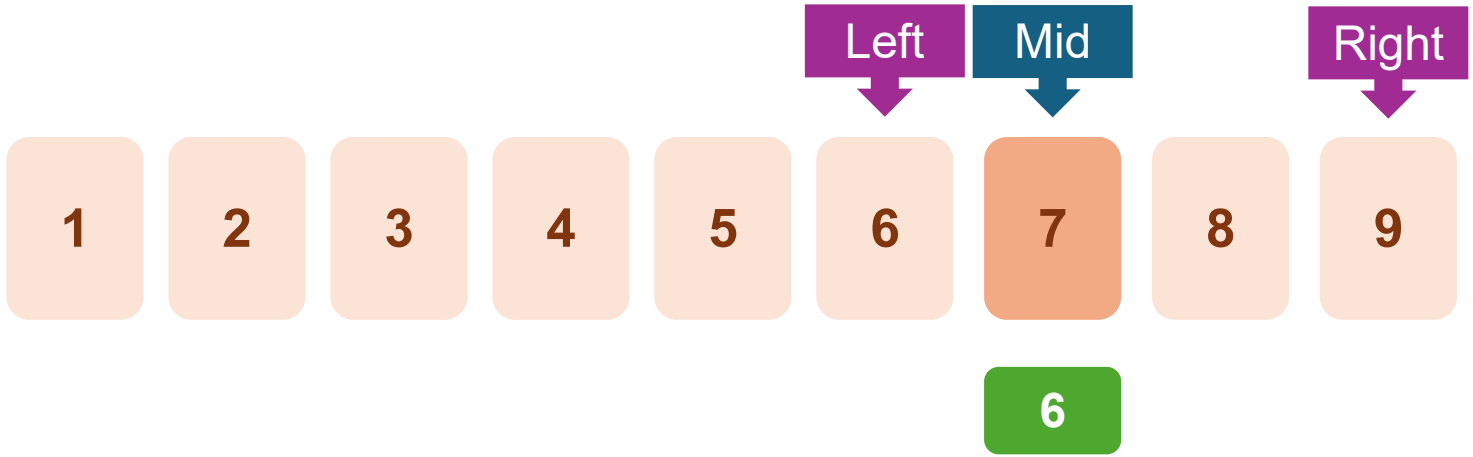
Tìm kiếm nhị phân

Ví dụ: Tìm trong dãy sau phần tử có giá trị là 6



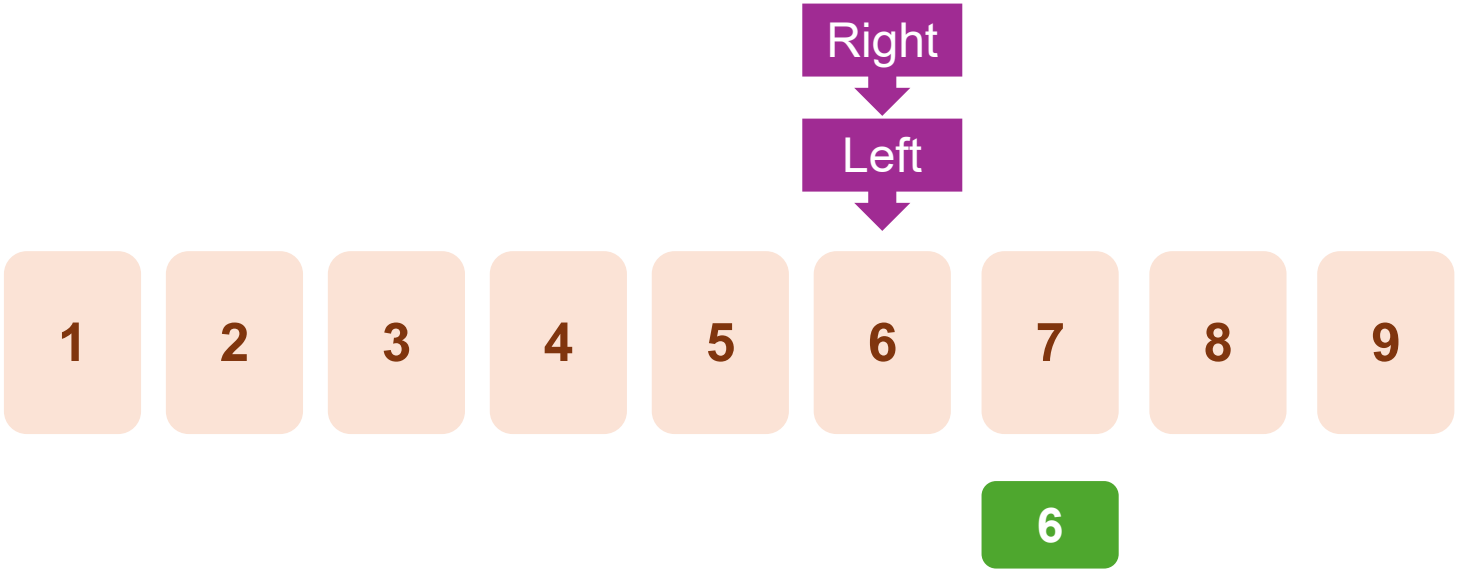
Tìm kiếm nhị phân

Ví dụ: Tìm trong dãy sau phần tử có giá trị là 6



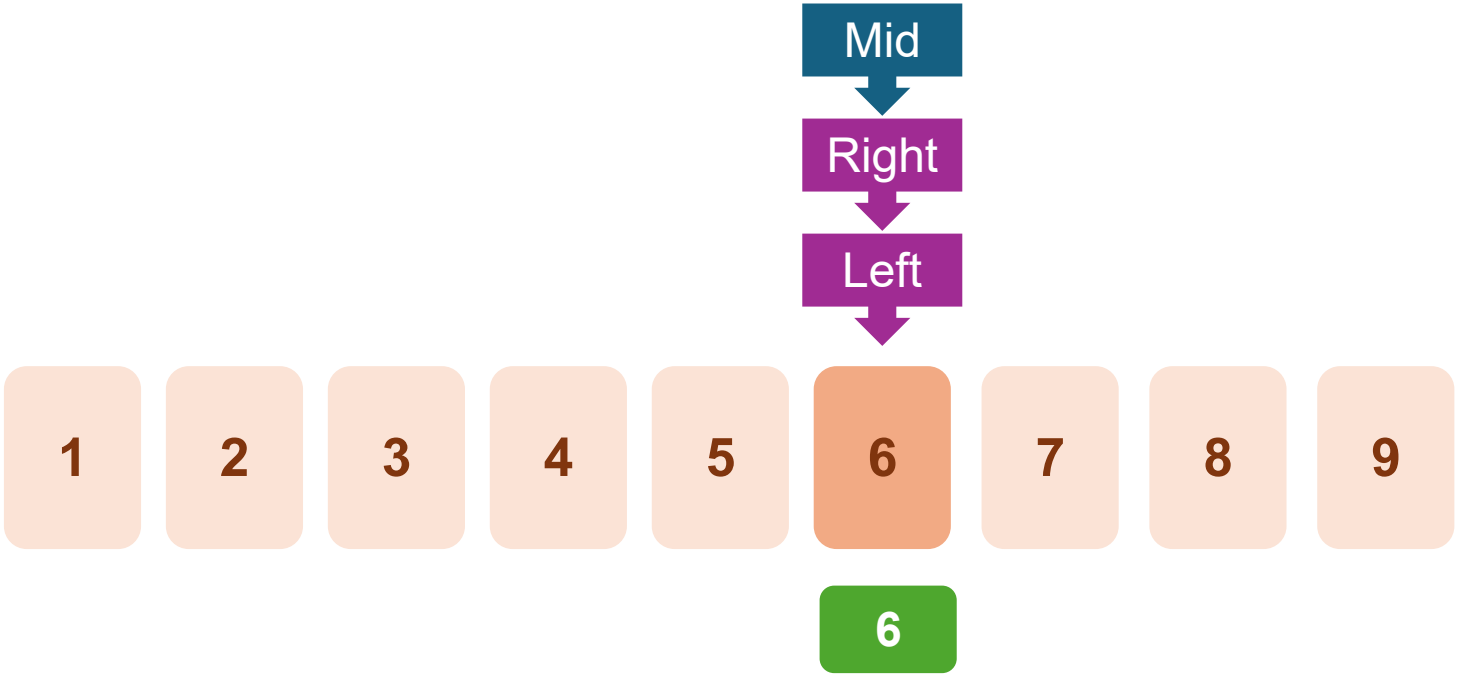
Tìm kiếm nhị phân

Ví dụ: Tìm trong dãy sau phần tử có giá trị là 6



Tìm kiếm nhị phân

Ví dụ: Tìm trong dãy sau phần tử có giá trị là 6



Tìm kiếm nhị phân

Ví dụ: Tìm trong dãy sau phần tử có giá trị là 6



BÀI TẬP CHƯƠNG 4

Bài 1. Trình bày sự thay đổi của dãy số sau nếu nó được sắp xếp tăng dần:

28, 17, 22, 9, 24, 11, 30, 21, 8, 4, 10

A. Sử dụng Merge Sort

B. Sử dụng Quick Sort Lomuto Partition với chốt ở cuối dãy.

C. Sử dụng Quick Sort Hoare Partition với chốt ở đầu dãy.

D. Sử dụng Quick Sort Lomuto Partition với chốt ở giữa dãy.

Bài 2. Đếm số cặp nghịch thế có trong một mảng (trong trường hợp sắp xếp tăng)

Bài 3. Có n vị trí và c con bò. Hãy đặt các con bò vào các vị trí sao cho khoảng cách nhỏ nhất giữa hai con bò bất kỳ là lớn nhất có thể.

Gợi ý: *Tìm kiếm nhị phân trên giá trị "khoảng cách nhỏ nhất".*

THANKS